Глава 3.
Классы языка C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

3.1 Описание класса и его компонентов

Синтаксис описания класса

[Атрибуты][Спецификаторы]`class` Имя: Предки Тело_класса

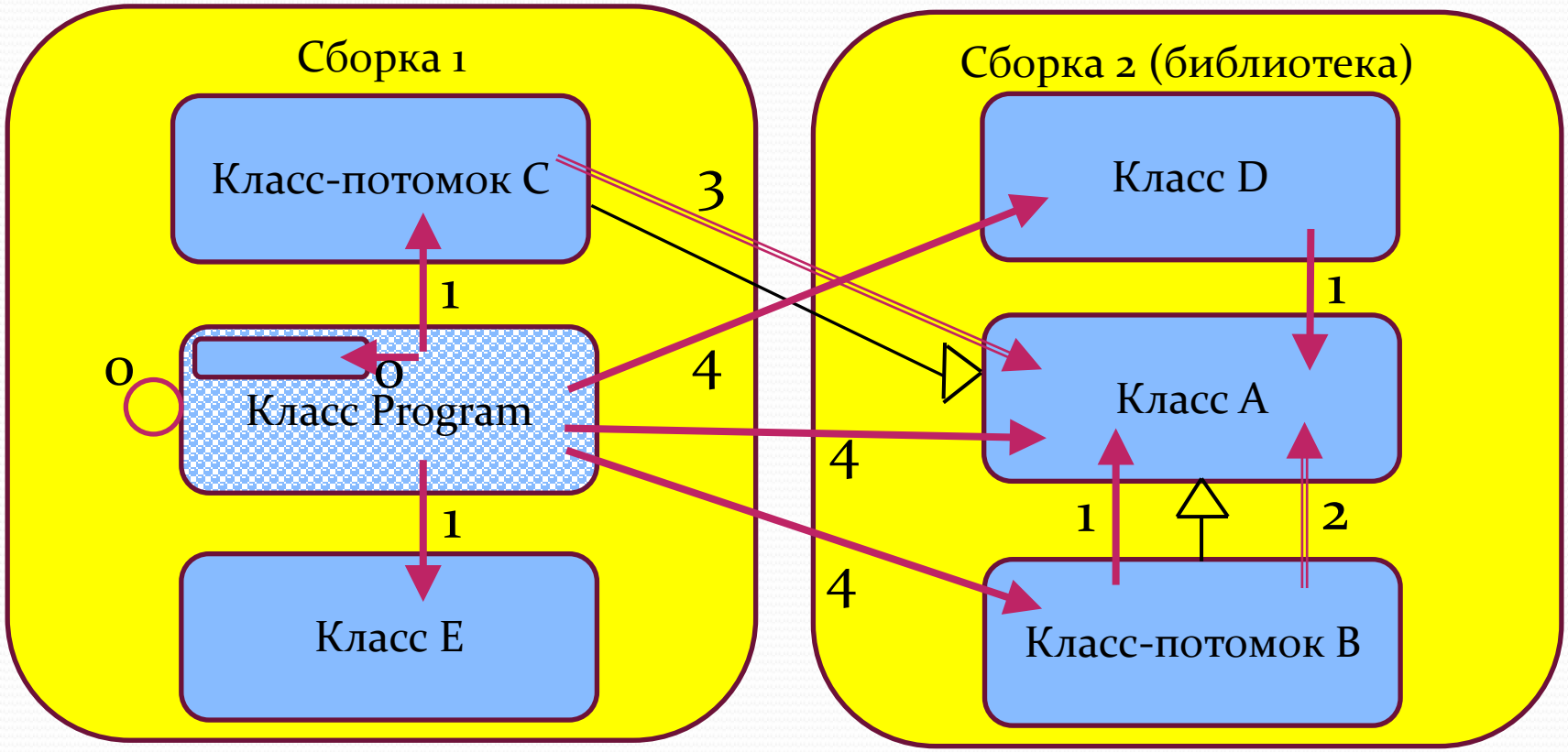
Спецификатор	Описание
`public`	Доступ не ограничен.
`protected`	Доступ из производных классов.
`private`	Доступ только из данного класса. *Подразумевается по умолчанию для вложенных классов.*
`internal`	Доступ из классов сборки. *Подразумевается по умолчанию для всех классов, кроме вложенных.*
`protected internal`	Доступ из производных классов этой и других сборок, а также классов данной сборки.
`private protected`	Доступ из производных классов в пределах сборки.
`abstract`	Абстрактный класс.
`static`	Статический класс.
`new`	Для вложенных классов. Задает новое описание класса взамен унаследованного.
`sealed`	Бесплодный класс - исключается наследование от класса.

Спецификаторы доступа

Спецификаторы доступа применяются к классам, их элементам и структурам. Желтым отмечены доступные элементы, синим - недоступные.

Квалификатор	Текущий класс	Потомки из текущей сборки	Классы сборки и потомки из другой сборки	Классы сборки	Классы из другой сборки
`public`					
`protected internal`					
`internal`					
`protected`					
`private protected`					
`private`					

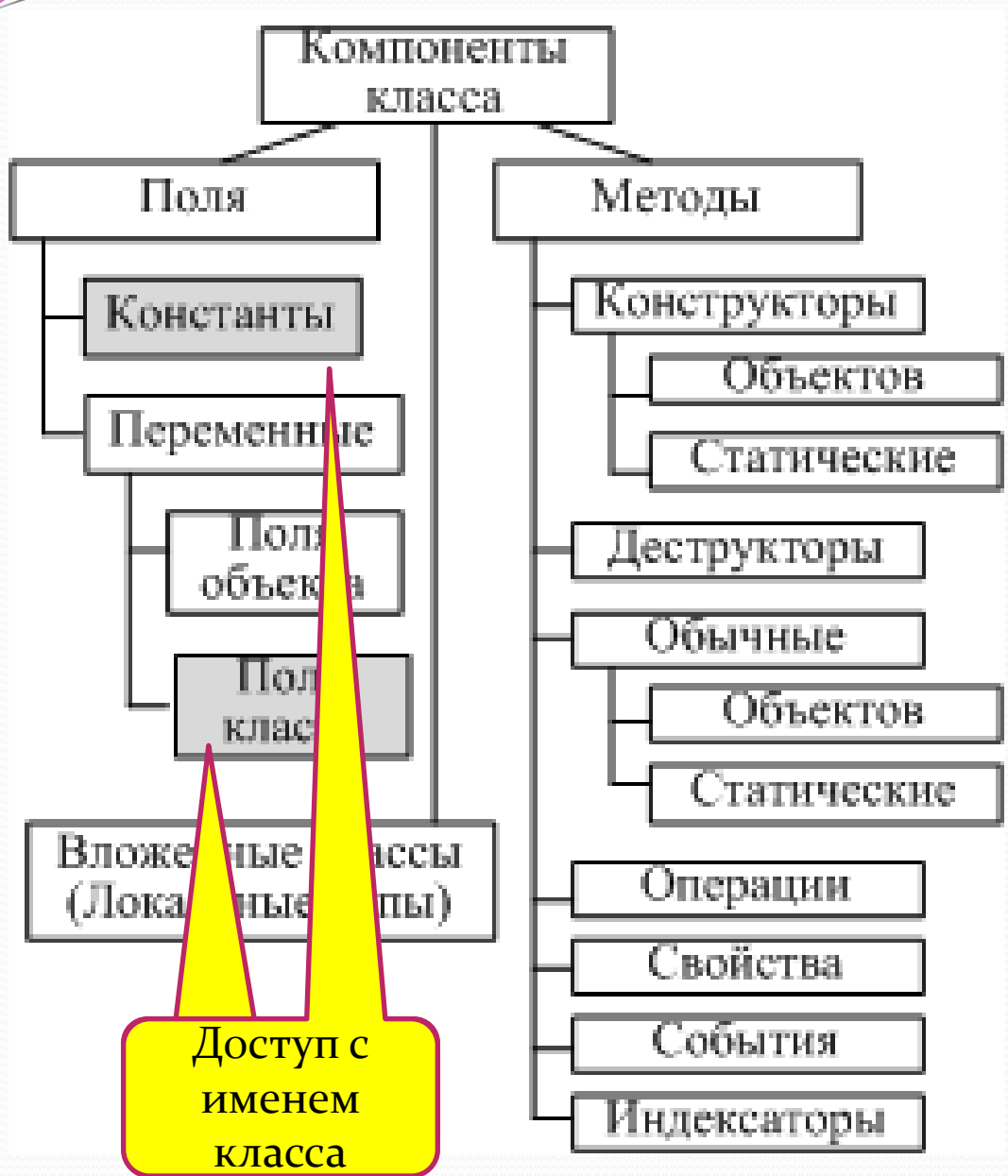
Пример Спецификаторы доступа



0 – внутри класса;
1 – внутри сборки;
2 – к базовому классу внутри сборки ;
3 – к базовому классу в другой сборке;
4 – к классу в другой сборке

private – возможны обращения 0;
internal – возможны обращения 0 и 1 ;
protected internal – возможны обращения 0 - 2 ;
private protected – возможны обращения 0 и 2;
protected – возможны обращения 0, 2 и 3;
public – возможны обращения 0 - 4

3.2 Компоненты класса



Компоненты класса:

- *поля* - хранят данные;
- *методы* – процедуры и функции, обрабатывающие данные.

Поля:

- *константы* – единственный набор на все объекты,
- *переменные*:
  - поля объекта – свой набор на каждый экземпляр,
  - поля класса или статические поля – единственный набор на все объекты.

Компоненты класса (2)

Методы:

- *конструкторы* – определяют действия по инициализации объектов:
  - конструкторы объектов,
  - статические конструкторы – для инициализации статических полей – вызывается один раз до 1-го обращения к полям и методам класса;
- *деструкторы* – определяют действия, которые должны быть выполнены перед уничтожением объектов,
- *операции* – задают действия над объектами с использованием операций,
- *обычные методы*, реализующие элементы поведения объектов:
  - методы объекта,
  - методы класса (статические методы);
- *свойства* – характеристики объектов вместе со способами их записи и чтения;
- *события* – определяют уведомления, которые может генерировать класс;
- *индексаторы* – обеспечивают доступ к элементам класса по их порядковому номеру.

Для всех методов возможна параметрическая перегрузка.

3.3 Поля

Формат описания полей в классе:

[Атрибуты] [Спецификаторы] Тип Имя[=Значение]

Спецификатор	Описание
`public`	Доступ не ограничен.
`protected internal`	Доступ из методов данного и производных классов, а также методов других классов данной сборки.
`internal`	Доступ из методов классов данной сборки.
`protected`	Доступ из методов данного и производных классов.
`private protected`	Доступ из методов данного и производных классов в пределах сборки.
`private`	Доступ из методов данного класса. *Подразумевается по умолчанию.*
`const`	Неизменяемое значение, инициализируется при компиляции программы, единственное для всех объектов, возможно обращение через имя класса.
`readonly`	Доступ только для чтения, инициализируется при компиляции или конструктором.
`static`	Единственное для всех экземпляров класса, обращение через имя класса.
`new`	Задает новое описание поля, скрывающее унаследованное.
`volatile`	Значение может меняться другим процессом.

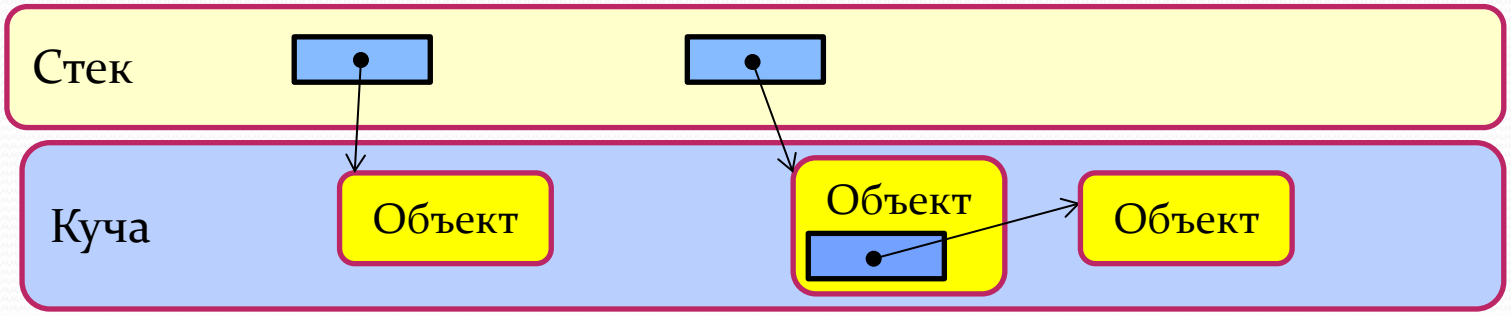
Пример

Ex3.01

```
using System;
namespace Ex3_01
{
  class A
  {
      private int i=1;                  // скрытое поле объекта
      double z;                         // скрытое поле объекта (по умолчанию)
      public int v;                             // общедоступное поле объекта
      public const double w = 32.517;   // константное поле класса
      public static string s = "D";     // статическое поле класса
  }
…
}
```

3.4 Создание объектов

Объекты – данные ссылочного типа, т.е. сами данные хранятся в куче, а ссылки на них – преимущественно в стеке. Исключение, если ссылка – часть данных другого ссылочного типа.



Объекты создаются:

- системой;
- программистом с использованием операции new.

Если конструкторы в классе не объявлены, то автоматически создается пустой конструктор без параметров.

Примеры:

```
A a = new A();          // вызывается конструктор без параметров
A b = new A(5,7,2);     // вызывается конструктор с параметрами
```

Пример создания объекта и доступа к полям

Ex3.01

```
using System;
namespace Ex3_01
{
   class A
  {
       private int i=1;                    // скрытое поле объекта
       double z;                           // скрытое поле объекта
       public int v;                       // общедоступное поле объекта
       public const double w = 32.517;     // константное поле класса
       public static string s = "D";       // статическое поле класса
  }
  class Program
  {
       static void Main()
       {
              A a = new A();               // создание объекта
              Console.WriteLine(a.v);      // доступ через имя объекта
              Console.WriteLine(A.w);      // доступ через имя класса
              Console.WriteLine(A.s);      // доступ через имя класса
           // Console.WriteLine(a.z);      // ошибка, доступ невозможен!
       }
  }
}
```

3.5 Методы

Синтаксис описания методов:

[Атрибуты] [Спецификаторы] Тип Имя([Параметры]) Блок

Спецификатор	Описание
`public`	Доступ не ограничен.
`protected internal`	Доступ из методов данного и производных классов, а также методов других классов данной сборки.
`internal`	Доступ из методов классов данной сборки.
`protected`	Доступ из методов данного и производных классов.
`private protected`	Доступ из методов данного и производных классов данной сборки.
`private`	Доступ из методов данного класса. *Подразумевается по умолчанию.*
`static`	Не получает адреса полей объектов. Вызов через имя класса.
`new`	Задает новое описание метода, скрывающее унаследованное.
`virtual`	Виртуальный
`override`	Перекрывающий виртуальный
`abstract`	Абстрактный
`extern`	Внешний
`sealed`	Бесплодный (нельзя переопределять виртуальный метод при наследов.)

Параметр this

В любой *нестатический* метод неявно передается параметр this – адрес (ссылка) полей объекта.

В явном виде this используют, если :

- метод возвращает объект класса;
- имена формального параметра метода и поля класса совпадают.

Пример:

```
class Demo
{
  double y;
  public Demo T()  //метод возвращает ссылку на объект
  { return this; }
  public void Sety( double y )
  { this.y = y; }  // имена формального параметра и поля класса
                   // совпадают
}
```

3.5.1 Параметры методов

Используется 4 типа параметров:

- *параметры-значения*, описанные без указания ключевого слова;
- *параметры-ссылки*, описанные **`ref`**;
- *выходные параметры*, описанные **`out`**;
- *параметры массивы*, описанные **`params`** – используются при переменном количестве параметров, объявляется последним.

Формальные и фактические параметры (аргументы) должны соответствовать по количеству, типу и порядку следования.

Аргументы параметров-значений должны иметь тот же тип или тип, для которого существует преобразование к типу формального параметра.

Если параметр определен, как параметр-значение, то при вызове метода в качестве аргументов можно указывать литералы и выражения совместимых типов.

Возможно задание параметров по умолчанию.

Возможно задание поименованных аргументов при вызове метода.

Параметры по умолчанию и поименованные аргументы

Также возможно объявление параметров метода по умолчанию, например:

```
void PrintPerson(string name, int age = 1,
                                 string company = "Undefined")
{
    Console.WriteLine($"Name: {name}
                          Age: {age}  Company: {company}");
}
```

Примеры вызовов:

```
PrintPerson("Tom", 37, "Microsoft");
PrintPerson("Tom", 37);
PrintPerson("Tom");
```

Порядок задания произвольный

или через поименованные аргументы:

```
PrintPerson("Tom", company:"Microsoft", age: 37);
PrintPerson(age:41, name: "Bob");
PrintPerson(company:"Google", name:"Sam");
```

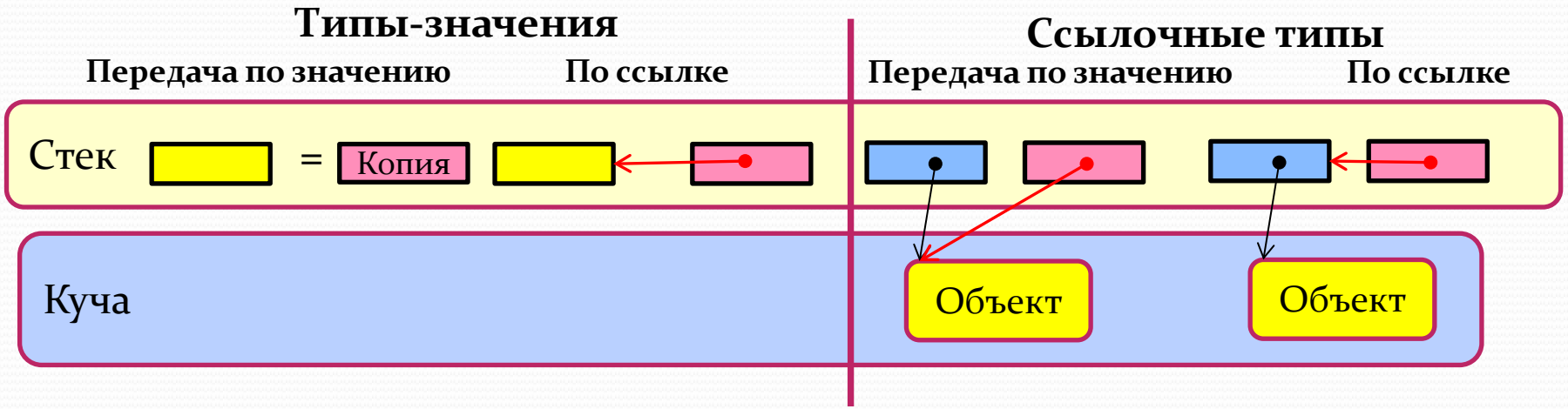
Описание параметров

Формат описания параметров:

Тип_параметра Тип_данных Имя_формального_параметра

Тип параметра	Служебное слово	Описание
Значение	—	*Передача по значению*: передается копия параметра. В качестве фактического значения можно указывать константы, переменные, литералы и выражения совместимых типов.
Ссылка	**`ref`**	*Передача по ссылке*: передается адрес данных, данные должны быть инициализированы.
Выходной	**`out`**	*Передача по ссылке*: передается адрес данных, данные могут быть не инициализированы. Компилятор следит, чтобы значение параметру было присвоено внутри метода. При вызове перед этим параметром также ставится out.
Переменное количество параметров	**`params`**	Передача заранее неизвестного количества параметров. Ставится в конце перечня параметров и задает массив параметров неопределенной длины

3.5.2 Передача типов-значений и ссылочных типов по значению и по ссылке



При передаче типа-значения в метод по значению в стеке строится его *копия*.

При передаче типа-значения в метод по ссылке в стек записывается его *адрес*.

При передаче в метод объектов ссылочного типа по значению передается *копия ссылки*. При этом возможно изменение полей объектов по переданному адресу, но, если создается новый объект, то в вызывающем методе *ссылка не меняется*.

При передаче ссылочного типа по ссылке передается *ссылка на ссылку*, т.е. возможно *замена объекта внутри метода*.

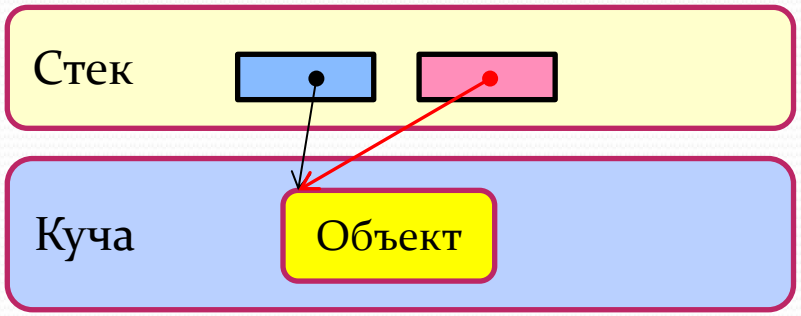
Пример

Ex3.02

```
class Ball
{
  public int Diameter;
}
class Program
{
    static void InflateBall(Ball ball)  // изменит объект
    {
        ball.Diameter++;
    }
```

Конструктор без параметров создастся по умолчанию!

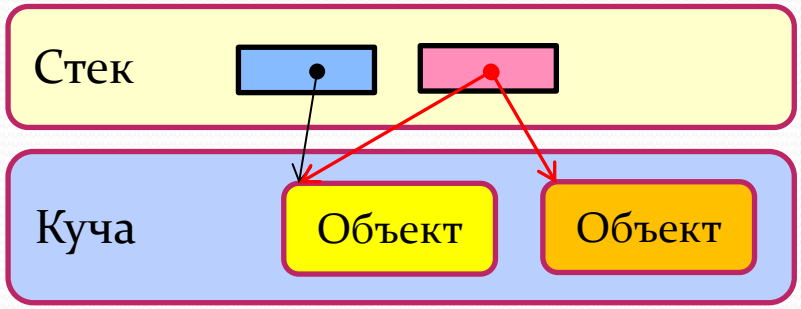
Передача по значению



Пример (2)

```
static void GetOldBall(Ball ball)  // не изменит адрес!
{
    ball = new Ball();
    ball.Diameter = 1;
}
```

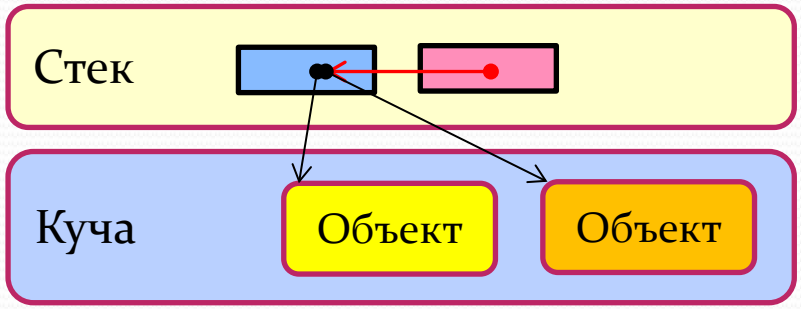
Передача по значению



Пример (3)

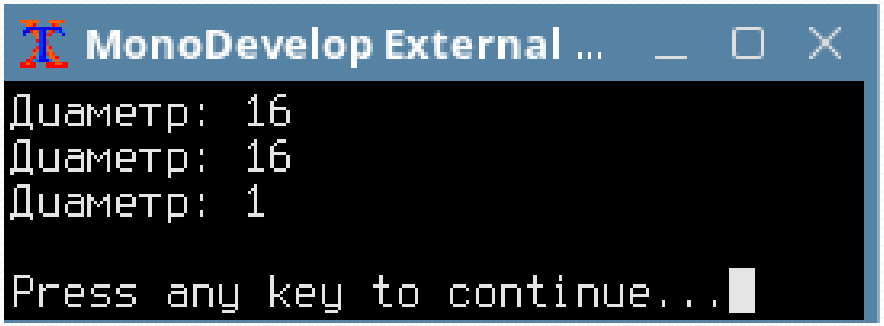
```
static void GetNewBall(ref Ball ball)  // изменит адрес!
{
    ball = new Ball();
    ball.Diameter = 1;
}
```

Передача по значению



Пример (окончание)

```
static void Main(string[] args)
{
    Ball football = new Ball();          // создание объекта
    football.Diameter = 15;              // инициализация поля
    InflateBall(football);
    Console.WriteLine("Диаметр: {0}",
                                  football.Diameter);
    GetOldBall(football);
    Console.WriteLine("Диаметр: {0}",
                                  football.Diameter);
    GetNewBall(ref football);
    Console.WriteLine("Диаметр: {0}",
                                  football.Diameter);
}
}
```



3.5.3 Выходные параметры

При необходимости метод может возвращать дополнительные значения через выходные параметры. Им обязательно должно присваиваться значение внутри метода.

Возвращает площадь

Ex3.03

Пример:

```
static int CalculateRectangle(int width, int height,
                                                out int perimeter)
{
     perimeter = (width * 2) + (height * 2);
     return width * height;
}
static void Main(string[] args)
{
     int area;
     int perimeter;
     area = CalculateRectangle(5, 10, out perimeter);
     Console.WriteLine("Area: {0}", area);
     Console.WriteLine("Perimeter: {0}", perimeter);
}
```

Дополнительно возвращает периметр

3.5.4 Переменное количество параметров

Передача заранее неизвестного количества параметров. Ставится в конце перечня параметров и задает массив параметров неопределенной длины.

Размер массива определяется свойством Length. Обращение выполняется по правилам массивов.

Пример:

```
public int Calculate(int a,out int c,params int[] d)…
```

3.6 Конструкторы

Конструктор – метод, автоматически вызываемый для инициализации полей при создании объекта с помощью new. Если явно конструктор не описан, то автоматически создается пустой *конструктор без параметров*, который инициализирует поля-значения нулями, а поля ссылочного типа значением null.

Формат описания:

Имя_класса ([Параметры])[: Инициализатор] Блок

Конструктор:

- не возвращает значения (даже void);
- может быть несколько раз параметрически перегружен, т.е. конструкторы должны иметь различные *сигнатуры*;
- без параметров называется *конструктором по умолчанию;*
- при отсутствии инициализатора базового класса автоматически вызывает конструктор базового класса.

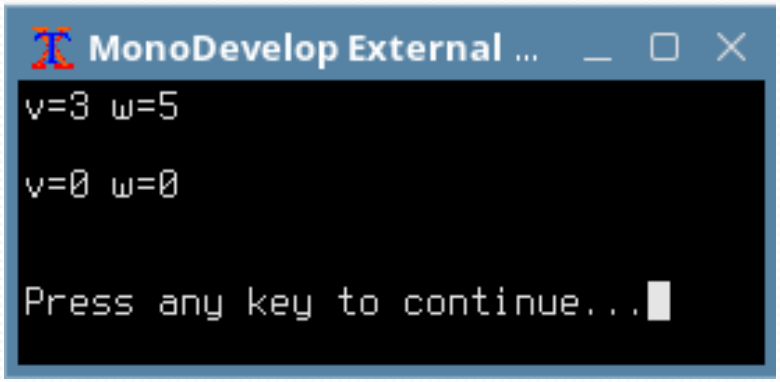
При необходимости инициализации полей базового класса конструктор базового класса в инициализаторе вызывается явно с параметрами:

`base`(Аргументы)

Пример класса с конструкторами

Ex3.04

```
using System;
namespace Ex3_04
{
    class A
    {
        public int v, w;
        public A(int _v,int _w){  v = _v;  w = _w; }
        public A(){}
    }
    class Program
    {
        static int Main()
        {
            A a = new A(3,5),b = new A();
            Console.WriteLine("v={0} w={1}\n",a.v,a.w);
            Console.WriteLine("v={0} w={1}\n",b.v,b.w);
        }
    }
}
```



3.7 Статические поля и методы. Статические классы

Статические поля – единственные для всех экземпляров класса.

Статические методы не получают параметра this.

Если класс содержит только статические компоненты, то объекты класса создавать запрещено. Такой класс объявляют *статическим*.

Инициализировать статические поля класса можно при его объявлении или в статическом конструкторе.

Статический конструктор инициализирует статические поля класса. Он не имеет параметров и его нельзя вызвать явно. Вызов происходит автоматически до создания объектов класса или вызова первого его метода.

Пример статического класса

Ex3.05

```
using System;
namespace Ex3_05
{
    static class D
    {
        static int a;
        static double b = 0.002;
        public static void Print()
        {
            Console.WriteLine("a=" + a);
            Console.WriteLine("b=" + b);
        }
        static D()
        {   a = 2;   }
    }
    class Program
    {
        static void Main()
        {   D.Print();   }
    }
}
```

MonoDevelop Extern...

```
a=2
b=0,002

Press any key to continue...
```

Инициализация статического поля при его объявлении

Инициализация статического поля статическим конструктором

Вызываем метод с именем класса, объект не создан

3.8 Свойства (простые)

Свойства – определяют *интерфейс* к закрытым полям класса. Таким образом можно не просто читать или писать значение поля, а выполнять дополнительные проверки или операции, например, устанавливать некоторые признаки.

Формат описания:

[**Атрибуты**][**Спецификаторы**] **Тип Имя** {

[**`get` Метод_чтения**]

[**`set` Метод_записи**]

} [= **Инициализирующее_значение**]

Значения спецификаторов совпадают со значениями спецификаторов методов.

При объявлении может отсутствовать либо get, либо set, тогда соответственно невозможно чтение свойства или его запись.

При обращении к свойству методы чтения или записи вызываются автоматически. Метод get – функция, возвращающая значение посредством return, метод set – процедура с параметром value.

Примеры

```
1)  private int _property1 = 0;  // поле
    public int property1          // объявление свойства
    {
        get { return _property1; }   // метод чтения
        set { _property1 = value; }  // метод записи
    }

2) public string property2 { get; set; }  // поле и методы
                                  // чтения-записи создаются автоматически

3) public float property3 { get; private set; } = 34.24;
                        // поле и методы чтения-записи создаются автоматически
```

Запись свойства: `a.property1 = 334;` // a – имя объекта

Чтение свойства: `int temp = a.property1;`

Примеры

Ex3.06

```
using System;
namespace Ex3_06{
  class A  {
      private int _x = 5;        // поле свойства
      public int X               // объявление свойства
      { get { return _x; }
        set { _x = value; }
      }
  }
    class Program  {
       static void Main() {
            A a = new A();
            Console.WriteLine("x= {0}, a.X);
             a.X = 6;
            Console.WriteLine("x= {0}, a.X);
       }
    }
}
```

или
`public int X {get; set;}`